Автономная некоммерческая организация высшего образования
«Университет Иннополис»

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
(БАКАЛАВРСКАЯ РАБОТА)
по направлению подготовки
09.03.01 - «Информатика и вычислительная техника»

GRADUATION THESIS
(BACHELOR'S GRADUATION THESIS)
Field of Study
09.03.01 – «Computer Science»

Направленность (профиль) образовательной программы
«Информатика и вычислительная техника»
Area of Specialization / Academic Program Title:
«Computer Science»

Тема / Topic	**Проверка семантической согласованности разнородных аналитических артефактов с использованием больших языковых моделей / Semantic consistency checking of heterogeneous analytical artifacts using large language models**

Работу выполнил / Thesis is executed by	**Загурских Елизавета Сергеевна / Zagurskikh Elizaveta**	подпись / signature
Руководитель выпускной квалификационной работы / Supervisor of Graduation Thesis	**Садовых Андрей Александрович / Sadovykh Andrey**	подпись / signature
Консультанты / Consultants		подпись / signature

Иннополис, Innopolis, 2027

Contents

1 Introduction **7**

1.1 Background and Motivation 7

1.2 Research Problem and Gap 8

1.3 Research Aim and Questions 9

1.4 Proposed Approach and Evaluation 11

1.5 Expected Contribution and Thesis Structure 12

2 Literature Review **13**

2.1 Requirements Traceability: Concepts and Representations . . . 13

2.2 Automated Traceability Link Recovery 17

2.3 Traceability-Based Consistency Analysis and Context Selection . 22

2.4 LLM-Based Semantic Consistency Checking 27

2.5 Research Synthesis and Gap 33

3 Methodology **37**

4 Implementation **38**

5 Evaluation and Discussion **39**

6 Conclusion **40**

Bibliography cited **41**

Abstract

My abstract starts from here.

Chapter 1

Introduction

1.1 Background and Motivation

Software development relies on analysis artifacts that describe stakeholder needs, expected system behavior, and the processes in which a system operates. These artifacts capture different views of a proposed solution. Functional requirements (FRs) specify the capabilities and constraints of a system, use cases (UCs) describe interactions between actors and the system, and Business Process Model and Notation (BPMN) models represent activities, responsibilities, decisions, and execution order. Requirements engineering standards emphasize the need to develop, manage, and maintain requirements throughout the system life cycle [1]. In practice, however, related information may be distributed across documents and models created by different analysts and maintained at different times.

The same functionality may therefore be expressed in several forms and at different levels of detail. For example, a functional requirement may state that a customer's identity must be verified before an application is approved. A use case may describe the verification as part of an interaction with the customer,

while a BPMN model may represent the same operation as a task preceding a decision gateway. These descriptions need not use identical terminology or contain identical details. Nevertheless, they should remain compatible regarding the required behavior, responsible actors, conditions, and order of activities. If one artifact changes without corresponding revisions to related artifacts, their descriptions may diverge. Such inconsistencies can create ambiguity for implementation, testing, and subsequent maintenance.

Traceability helps analysts identify related information across artifacts. Research on requirements traceability has established the importance of representing and maintaining links among development objects [2]. However, a link indicates a relationship, not necessarily agreement between the linked descriptions. Two elements may refer to the same operation while prescribing different execution conditions or assigning responsibility to different actors. Consequently, checking the existence of trace links and checking the semantic consistency of linked elements are distinct tasks.

1.2 Research Problem and Gap

Consistency checking becomes particularly challenging when artifacts differ in notation, structure, and level of abstraction. A systematic literature review of cross-domain model consistency checking identifies interoperability, dependency management, and the detection of behavioral inconsistencies among the continuing challenges in this area [3]. Rule-based techniques can examine explicitly formalized constraints, but natural-language requirements and use case descriptions often contain information that is not represented as formal rules. Moreover, evaluating a single pair of elements may be insufficient when its meaning depends

on surrounding process steps, preconditions, or alternative flows.

Recent research has investigated automated consistency checking in several related settings. Keim et al. use traceability link recovery to identify missing and undocumented elements across software architecture documentation and models [4]. Gärtner and Göhlich combine a large language model (LLM) with formal logic to detect contradictions between requirements written in controlled natural language [5]. Schulte et al. apply LLMs to coherence checking across levels of business process documentation [6]. Dietrich et al. develop an LLM-based approach to detecting structural and semantic inconsistencies among programming exercise artifacts [7]. These studies demonstrate the potential of automated analysis, while addressing different artifact combinations, inconsistency definitions, and evaluation settings.

The remaining question for this thesis is how reliably an LLM can identify *behavioral inconsistencies between heterogeneous analysis artifacts*, specifically FRs, UCs, and BPMN elements. A related question concerns the information supplied to the model. A direct comparison of two related elements may omit conditions or dependencies needed for interpretation, whereas adding neighboring artifact elements may supply useful evidence or irrelevant detail. Existing results do not establish which bounded context is most useful for consistency decisions across the selected artifact pairs. The thesis therefore examines the effect of local structural and traceability-related context under controlled comparison conditions.

1.3 Research Aim and Questions

The aim of this thesis is to develop and evaluate a structured LLM-based method for detecting semantic and behavioral inconsistencies across heteroge-

neous business and systems analysis artifacts. The study focuses on candidate relationships between FRs and UCs, FRs and BPMN elements, and UCs and BPMN elements. Candidate related elements are supplied as input; discovering all possible trace links or classifying link types is outside the primary scope. Where necessary, a requirement may correspond to several process tasks rather than to a single isolated task.

The study addresses three research questions:

RQ1. How accurately can an LLM detect and classify behavioral inconsistencies between related FR, UC, and BPMN elements?

RQ2. How does the inclusion of local structural context and traceability-related neighboring elements affect detection performance compared with checking the target pair alone?

RQ3. How does detection performance vary across artifact-pair types, inconsistency categories, and repeated model runs?

The analysis considers four inconsistency categories: *missing behavior*, when required functionality is absent from the corresponding element or set of elements; *actor or responsibility mismatch*, when artifacts assign incompatible responsibilities; *condition or order mismatch*, when prerequisites, decisions, or activity sequences disagree; and *semantic deviation*, when related elements express materially different intended behavior. The categories provide a common basis for annotation and evaluation while allowing the same underlying functionality to appear in different representations.

1.4 Proposed Approach and Evaluation

The proposed method represents relevant artifact elements and their candidate correspondences in a structured form. It then presents target elements to an LLM with an explicit consistency-checking task and a constrained output format. Each decision is expected to indicate whether an inconsistency is present, identify its category when applicable, and provide supporting evidence from the supplied artifacts. The method is designed to support analyst review rather than replace expert judgment.

The experiment compares controlled input configurations that differ in the context available to the model. The simplest configuration contains the target elements. Additional configurations incorporate bounded information from the internal structure of an artifact, such as neighboring BPMN tasks and gateways or use case preconditions and alternative steps, and from verified traceability relationships to other relevant elements. The target pairs, inconsistency definitions, prompt structure, and evaluation criteria are kept comparable so that differences in performance can be examined in relation to the supplied context.

The evaluation will use selected analysis materials from publicly accessible projects and cases. Candidate correspondences and consistency labels will be reviewed to establish a reference set. Where suitable, controlled modifications will introduce specified inconsistency types; these modifications will be independently checked rather than treated as valid solely because they were generated. The study will measure precision, recall, and F1-score for inconsistency detection, with separate analysis by category and artifact-pair type. Repeated runs will help assess the stability of model decisions. This design is intended to distinguish the ability to recognize a genuine discrepancy from the tendency to report unsupported

discrepancies.

1.5 Expected Contribution and Thesis Structure

The expected contribution is an empirically evaluated procedure for consistency checking across three commonly used, structurally different analysis artifact types. In addition to the checking procedure, the study aims to provide an operational inconsistency taxonomy, documented rules for selecting bounded context, and comparative evidence concerning the benefits and limitations of contextual information. These findings may help analysts determine when an LLM-based consistency checker is useful and which evidence it should receive. The thesis does not assume that more context will necessarily improve detection accuracy; this effect is a subject of the experiment.

The remainder of the thesis reviews research on traceability, cross-artifact consistency, and LLM-based verification; defines the method and annotation procedure; describes the experimental data and comparison settings; presents and analyzes the evaluation results; and concludes with limitations and directions for further research.

Chapter 2

Literature Review

2.1 Requirements Traceability: Concepts and Representations

Requirements traceability has been studied as a means of establishing and maintaining relationships between requirements, their origins, and the artifacts produced throughout software development. Gotel and Finkelstein [8] identified traceability as a multifaceted problem and distinguished between pre-requirements specification (pre-RS) traceability, which concerns the origins and development of requirements, and post-requirements specification (post-RS) traceability, which concerns their relationships with subsequent development artifacts. This distinction illustrates that traceability extends beyond connecting requirements to implementation: it also involves preserving information about their refinement, interpretation, and relationships across development activities [8], [9].

The organization of traceability information has received considerable attention in empirical research. Ramesh and Jarke [2] investigated traceability

practices in 26 software development organizations and identified substantial differences in how traceability information was captured and used. Their findings distinguish between relatively limited traceability practices and more extensive approaches that preserve additional information about dependencies, decisions, and development activities. The authors proposed reference models to describe these practices and support the organization of traceability information [2]. This work demonstrates that the usefulness of traceability depends not only on whether links exist, but also on the information represented by those links and the activities they are intended to support.

Traceability becomes particularly challenging when related information is distributed across artifacts with different structures and levels of abstraction. Winkler and von Pilgrim [10] surveyed traceability approaches in requirements engineering and model-driven development highlighting the diversity of artifact representations and the difficulties of establishing and maintaining relationships between them. While textual requirements express information primarily through natural language, model-based artifacts may encode additional information through graphical structures and modeling-language constructs. Consequently identifying a relationship between two artifacts does not necessarily imply that their elements can be compared through direct textual similarity [10], [11].

A related concern is the granularity at which traceability relationships are established. Links may connect complete documents, individual requirements, model elements, or more detailed units of information, depending on the traceability objectives and the available representations [10], [11]. Coarse-grained links can indicate that two artifacts are related, but they provide limited information about which particular statements or model elements correspond. Fine-grained links allow relationships to be examined at a more specific level, although they

also introduce additional complexity in link creation and maintenance [9], [12]. Moreover, a single requirement may correspond to several elements of a model, while one model element may contribute to multiple requirements. A representation of traceability must therefore be capable of accommodating relationships that are not restricted to one-to-one correspondences [10], [11].

Traceability information may be organized using requirements traceability matrices, repositories, or graph-based representations. A traceability matrix provides a tabular view of relationships between selected sets of artifacts. A graph-based representationin contrast, expresses artifact elements as nodes and their relationships as edges, making it possible to navigate connections involving multiple artifact types [10], [11]. These representations are not mutually exclusive: a matrix may be produced as a view of a more comprehensive collection of traceability relationships. The appropriate representation depends on the types of relationships being maintained and the analyses that the traceability information is expected to support [2], [9].

The practical value of traceability also depends on the correctness and maintenance of established links. Cleland-Huang et al. [9] note that traceability is frequently established in an ad hoc manner or retrospectively, limiting the extent to which its potential benefits are realized. Mäder and Gotel [12] address traceability maintenance by investigating how relationships between requirements and UML models can be updated as software artifacts evolve. More recently, Ruiz et al. [13] examined barriers to the adoption and use of traceability in practice, demonstrating that maintaining traceability remains an organizational and technical concern. These studies suggest that traceability should be treated as information that requires continuing validation rather than as a static collection of links.

The distinction between automatically proposed and validated trace links is particularly relevant to the reliability of downstream analyses. Maro et al. [14] investigated the information needed by human analysts when evaluating automatically generated trace links. Their findings indicate that the content of connected artifacts plays an important role in link validation, while additional contextual information does not necessarily provide equivalent benefits. This demonstrates that traceability quality cannot be inferred solely from the availability of relationship information. The evidence supporting a link and the correctness of its interpretation must also be considered [14], [15].

An important limitation follows from the distinction between identifying related artifacts and determining whether their contents agree. Traceability establishes a relationship between elements, but that relationship does not itself constitute evidence that the elements express compatible behavior. For example, a textual description and a corresponding process model may refer to the same activity while differing in the order of execution or omitting behavior present in the other representation. Such discrepancies have been investigated through consistency analysis of textual and model-based process descriptions [16]. Similarly, Keim et al. [4] demonstrate that recovered traceability relationships can provide a foundation for detecting missing or undocumented elements across software architecture representations.

These findings establish two complementary roles for traceability. First, it provides explicit correspondences that identify which artifact elements should be considered together. Second, it can support further analyses that assess the completeness or consistency of the information expressed by those elements [4], [16]. Howeverthe existence of a link and the semantic validity of the linked contents remain distinct properties. This distinction motivates the examination of

traceability not only as a relationship-recovery problem, but also as a source of information for cross-artifact consistency analysis.

2.2 Automated Traceability Link Recovery

Automated traceability link recovery (TLR) addresses the problem of identifying potential relationships between software artifacts when explicit trace links are unavailable or incomplete. Research in this area has evolved from information retrieval (IR) techniques toward approaches that incorporate artifact structure, learned semantic representations, and large language models [17], [18]. These developments have expanded the information available for identifying correspondences, although the effectiveness of individual methods continues to depend on the types and representations of the artifacts being analyzed [19], [20].

Early automated TLR approaches primarily relied on textual similarities between artifacts. In 2002 Antoniol et al. [21] investigated the recovery of links between source code and documentation using vector-space and probabilistic information retrieval models. Their approach treated artifact content as retrievable textual information enabling candidate correspondences to be identified without requiring previously established links. Marcus and Maletic [22] extended this research direction in 2003 by applying Latent Semantic Indexing (LSI) to documentation-to-code traceability. By representing documents in a reduced semantic space, LSI provided an alternative to matching artifacts exclusively through their directly observable terms.

However, generating candidate links did not eliminate the need to determine which recommendations were correct. In 2006, Hayes et al. [23] investigated candidate link generation as a process involving both retrieval methods and human

analysts. Their RETRO approach incorporated analyst feedback and introduced measures for assessing the usefulness of generated link recommendations. This work established an important distinction between automatically recommending relationships and confirming their validity. Subsequent research continued to treat human participation as a relevant component of traceability recovery, rather than assuming that an automatically generated link could always be accepted without verification [17], [23].

During the following decade, researchers increasingly investigated information beyond artifact text. In 2013, Panichella et al. [24] proposed incorporating structural relationships into IR-based traceability recovery. Their method used developer-validated links to determine when structural information should influence additional link recommendations. This distinction was important because structural proximity could help identify additional correct correspondences when the underlying links were reliable, but could also propagate incorrect recommendations when they were not [24]. The study therefore demonstrated that structural information should not necessarily be incorporated indiscriminately. A systematic mapping study published by Borg et al. in 2014 [17] further consolidated research on IR-based trace recoveryexamining the methods and empirical evidence available at that time.

The development of deep learning introduced another research direction. In 2017, Guo et al. [25] proposed a neural approach that incorporated semantic information and domain knowledge into software traceability. Rather than relying exclusively on manually designed text-similarity measures, the approach used learned representations to capture relationships between artifacts. In 2018, Rath et al. [26] investigated machine-learning-based augmentation of incomplete traceability information in software repositories, using information associated with

project issues, commits, and development artifacts. These studies illustrate different ways in which learned models and repository information can supplement traditional retrieval methods.

A further development appeared in 2021, when Lin et al. [27] introduced T-BERTa traceability approach based on pre-trained BERT models and transfer learning. Earlier neural approaches were affected by the limited availability of labeled traceability datasets. T-BERT addressed this difficulty by transferring knowledge from a related software engineering task to trace link prediction [27]. This represented a change in how semantic representations were obtained: instead of learning them solely from a limited collection of traceability examples, the method could exploit knowledge acquired during pre-training and transfer learning. Nevertheless, the evaluated task remained trace link recovery rather than detecting contradictions between the contents of already linked artifacts.

In parallel with the development of general-purpose retrieval methods, researchers investigated the specific characteristics of model-based artifacts. Rasiman et al. [19] evaluated automated TLR in model-driven development using features derived from textual and process-related information. Their 2022 study examined a machine-learning classifier on three industrial datasets, demonstrating the relevance of artifact-specific information when applying traceability techniques beyond conventional requirements-to-code relationships.

The importance of artifact characteristics is particularly evident in studies involving BPMN. In research published in 2022, Lapeña et al. [28] investigated how the linguistic particularities of BPMN models could be incorporated into requirements-to-BPMN traceability recovery. Their approach adapted a general traceability technique to the characteristics of process models rather than treating them as ordinary text documents. The results indicated that incorporating these

particularities could improve the recovery of relevant links while maintaining precision [28].

A related study by Lapeña et al. [29], also published in 2022, introduced execution-trace information to enhance recovery between requirements and BPMN models. The proposed method used process-model fragments and selected execution traces to enrich the information available for matching. Importantly, the authors considered the relevance of the added execution information: indiscriminately using all available execution traces could introduce similar terms into unrelated model fragments and weaken their ability to distinguish relevant correspondences [29]. This finding suggests that the selection of supplementary information can be as important as its availability.

More recent research has investigated the use of large language models for automated traceability recovery. In 2024, Bonner et al. [30] evaluated approaches based on embedding similarity and LLM-based classification for establishing trace links between requirements and model-based systems engineering artifacts. Their findings emphasized the role of data preprocessing and model adaptation in improving the quality of recommendations [30].

In 2025, Fuchß et al. [20] introduced LiSSA, a framework that combines LLMs with retrieval-augmented generation (RAG) for traceability link recovery. Unlike approaches designed for a single artifact combination, LiSSA was evaluated on requirements-to-code, documentation-to-codeand architecture-documentation-to-model traceability tasks. The reported results showed improvements over existing approaches for the code-related tasks, but did not establish equally strong performance for all artifact combinations [20]. This variation is significant because it indicates that the capabilities of an LLM-based traceability approach cannot be assumed to transfer uniformly between artifact types.

The emergence of LLM-based methods has not eliminated alternative research directions. For example, Mosquera et al. [15] evaluated an ontology-based approach combining automatic reasoning and NLP for recovering links between software requirements and conceptual models. Their 2025 study also examined the involvement of human analysts in traceability discovery. Similarly, the systematic mapping study by Wan et al. [18] reviewed IR-based traceability research published between 2014 and 2024 and identified enhancement strategies involving artifact text, structural information, model-based optimizationand human intervention. These findings demonstrate that contemporary traceability research encompasses multiple complementary methodological directions rather than a single progression in which newer techniques entirely replace earlier ones.

Overall, the reviewed studies show that traceability recovery has progressed from textual similarity toward increasingly context-sensitive approaches that can exploit artifact structure, learned representations, domain informationand retrieved evidence [20], [24], [25], [27], [29]. However, the primary objective of TLR remains the identification of valid relationships between artifact elements. Even when a method successfully recovers a linkits evaluation does not necessarily determine whether the connected artifacts express mutually consistent behavior. Consequently, the findings on traceability recovery provide a foundation for selecting related information, but they do not directly resolve the separate problem of semantic consistency checking.

2.3 Traceability-Based Consistency Analysis and Context Selection

Although traceability link recovery identifies relationships between software artifacts, these relationships can also serve as a foundation for evaluating whether related representations contain compatible information. Such analysis requires more than establishing that two elements refer to the same concept or functionality: it must also consider whether the elements describe consistent properties, constraints, and behavior [4], [9]. Consequently, traceability-based consistency analysis extends the use of artifact relationships beyond correspondence identification toward the examination of discrepancies between connected representations.

Early research on cross-artifact alignment demonstrated how relationships between textual requirements and process models could be established using combinations of linguistic and structural information. In 2011, Thayasivam et al. [31] introduced ProcGap, an approach combining natural language processing, information retrieval, and semantic reasoning to map requirements onto domain-specific reference process models. The method incorporated knowledge about process-model concepts rather than relying exclusively on textual matching. Although its principal objective was establishing mappings between requirements and process elements, the approach illustrated how semantic information could support the analysis of heterogeneous business and software representations.

A more direct investigation of consistency appeared in the work of van der Aa et al. published in 2017 [16]. The authors proposed an automated method for detecting discrepancies between textual process descriptions and corresponding process models. Their approach combined linguistic analysis, semantic similarity, and activity-ordering information to establish correspondences between sentences

and model activities. These correspondences were then used to identify activities represented in a process model but absent from its textual descriptionas well as differences in the order of execution [16]. This study is particularly significant because it demonstrates that semantic matching and behavioral analysis can be combined to detect inconsistencies without relying on large language models.

The importance of fine-grained alignment was further investigated by Sànchez-Ferreres et al. in 2018 [32]. Their method aligned textual process descriptions with elements of graph-based process modelsincluding activities, events, and gateways. Compared with approaches limited primarily to activity matching, the inclusion of additional model constructs provided a more comprehensive representation of relationships between textual and graphical descriptions [32]. This is relevant to consistency analysis because process behavior may depend not only on individual activities but also on branching and event-related information expressed through model structure.

A related difficulty concerns the ambiguity of natural-language descriptions. In another 2018 study, van der Aa et al. [33] introduced behavioral spaces to represent possible interpretations of textual process specifications. Their approach supported compliance checking while accounting for behavioral ambiguity that could otherwise lead to misleading conclusions about whether a process execution satisfies a textual description. This work demonstrates that an apparent difference between representations does not necessarily constitute an inconsistency: the interpretation permitted by the text must also be considered [33].

Other approaches have addressed consistency by deriving one representation from another. For example, Aysolmaz et al. [34] proposed a semi-automated method for generating natural-language requirements documents from business process models. By constructing textual requirements using information already

captured in process modelsthe approach aimed to improve consistencycompleteness, and maintainability. However, consistency supported through artifact generation differs from the problem of comparing representations that were created independently [34]. Independently authored artifacts may introduce discrepancies that cannot be prevented simply by using a shared generation procedure.

The broader problem of checking consistency between behavioral and heterogeneous models has also been examined through systematic literature reviews. Muram et al. [35] surveyed software behavioral model consistency checking organizing existing research according to model types, consistency concepts checking techniques, and inconsistency handling. Torres et al. [3] subsequently reviewed cross-domain model consistency checking supported by model-management tools. Their findings highlighted limitations in the types of inconsistencies addressed by existing tools and identified further challenges in consistency maintenance across different modeling domains [3]. Together, these reviews demonstrate that consistency checking is an established research area, while also showing that supporting heterogeneous representations and different forms of inconsistency remains challenging.

Traceability information can provide a direct mechanism for identifying potential inconsistencies. In 2023, Keim et al. [4] presented an approach that uses traceability link recovery between natural-language software architecture documentation and architecture models. The recovered relationships form the basis for detecting model elements without corresponding textual documentation and elements described in the text but not represented in the model. This approach demonstrates a concrete connection between traceability recovery and downstream inconsistency detection [4]. However, its examined discrepancies primarily concern the presence or absence of corresponding architectural elements. They do

not encompass all behavioral differences that can occur between requirements, use-case scenarios and process models.

An important methodological implication of traceability-based analysis is that its conclusions depend on the quality of the underlying correspondence information. If a relevant relationship is missing or an incorrect relationship is accepted, a downstream checker may examine the wrong elements or interpret the absence of a match as an inconsistency. This dependency follows from approaches in which recovered links are subsequently used for discrepancy detection [4]. Research on traceability validation therefore provides relevant evidence for the design of consistency analyses. Maro et al. [14], for example, investigated the information required by human analysts when confirming automatically generated links, showing that the content of the connected artifacts was particularly valuable during validation.

Beyond establishing correct correspondences, consistency analysis requires decisions about which information should be available when evaluating a particular relationship. Previous traceability research demonstrates that structural information can be beneficial when it is introduced selectively. Panichella et al. [24] used developer-validated links to control the incorporation of structural information into link recovery. Their results indicated that verified relationships could help guide the use of additional information, whereas relying on incorrect links could reduce the quality of recommendations. Although their study concerned traceability recovery rather than semantic consistency checking it provides evidence that the relevance and reliability of contextual relationships are important considerations.

Similar observations arise in research involving BPMN models. Lapeña et al. [29] used execution traces to enrich process-model fragments during requirements-

to-BPMN traceability recovery. Their method required selecting relevant execution information rather than simply adding all available traces. Indiscriminate expansion could introduce similar information into unrelated fragments and reduce their discriminative properties [29]. This illustrates a broader problem in artifact analysis: increasing the amount of available information does not necessarily increase its usefulness for a particular decision.

Recent requirements-engineering research has investigated context organization in LLM-assisted analysis. In 2025, Fantechi et al. [36] combined rule-based requirements clustering with LLM-based inconsistency detection. The clustering stage organized requirements into smaller groups before they were examined by the language model, improving the reported detection performance [36]. In 2026, Poulsen et al. [37] investigated context-aware LLMs for requirements ambiguity detection and found benefits from incorporating domain knowledge. Although these studies address different requirements-analysis tasks, they suggest that the organization and relevance of contextual information can influence LLM judgments.

For cross-artifact consistency analysis, two conceptually different sources of context can be distinguished. Local structural context contains information from the representation surrounding a selected artifact element. For example, a process activity may need to be interpreted together with preceding activities, gateways, or subsequent branches, while a use-case step may depend on preconditions or alternative flows. Traceability-guided context, by contrast, introduces information from other artifact elements connected through established cross-artifact relationships. The first source supports interpretation within an artifact, whereas the second can provide additional constraints or behavioral descriptions originating from related artifacts.

The reviewed research provides reasons to consider both sources of information. Studies of process-model consistency demonstrate the importance of structural and ordering information [16], [32], while traceability research shows how established relationships can guide the selection of related artifact elements [4], [24]. At the same time, studies of contextualized analysis indicate that additional information must be selected carefully [29], [36], [37].

However, these findings do not directly establish how different sources of context affect LLM-based behavioral consistency checking across functional requirements, use cases, and BPMN models. In particular, the reviewed studies do not provide a controlled comparison between examining an individual linked pair, adding its local structural context, and incorporating information selected through validated traceability relationships. This distinction provides the basis for investigating traceability-guided context as an input to semantic consistency analysis, rather than treating traceability recovery and consistency checking as a single task.

2.4 LLM-Based Semantic Consistency Checking

The application of large language models (LLMs) to requirements analysis has introduced new opportunities for detecting inconsistencies expressed in natural language. Unlike methods based exclusively on lexical similarity, LLM-based approaches can examine the meaning of requirement statements and reason about potential conflicts between their descriptions [5], [38]. However, existing research demonstrates that the role assigned to an LLM varies considerably between approaches. Some methods use language models directly to classify inconsistencies, whereas others combine them with structured representations,

formal rules, graphs, or external verification mechanisms [5], [39], [40], [41].

An early investigation of LLM-based inconsistency detection was presented by Fantechi et al. in 2023 [38]. The authors explored the ability of ChatGPT, using GPT-3.5, to identify inconsistencies in natural-language requirements. Their preliminary evaluation indicated that an LLM could recognize inconsistency defects that were difficult for conventional NLP-based approaches to detect [38]. This work demonstrated the potential of language-model-based semantic interpretation for requirements analysis. However, the examined task concerned inconsistencies within natural-language requirementsrather than behavioral alignment between heterogeneous artifact types.

A subsequent direction combined LLMs with explicit logical reasoning. In 2024, Gärtner and Göhlich [5] introduced ALICE, an automated method for detecting contradictions in requirements expressed in controlled natural language. The approach combines linguistic preprocessing, formal logic, and language-model-based analysis. In particular, ALICE uses a decision procedure organized around seven questions to evaluate whether two requirement statements are contradictory [5].

The authors reported that the combined approach outperformed a purely LLM-based comparison on the evaluated requirements datasets [5]. The improvement can be understood in relation to the method's design: instead of requiring the language model to make an unrestricted judgment about contradiction, the approach decomposes the decision into more specific linguistic and logical conditions. Nevertheless, the method depends on the interpretation of controlled natural-language requirements and the correct extraction of their constituent expressions. Its evaluation does not establish performance for independently authored requirements, use-case scenarios, and BPMN models.

The organization of information provided to an LLM became an explicit research concern in later work. In 2025, Fantechi et al. [36] investigated a combined method that applies rule-based requirements clustering before LLM-based inconsistency detection. Rather than processing a large requirements collection without prior organization, the method identifies smaller groups of requirements for analysis. The authors reported improvements in precision and recall compared with the evaluated alternatives [36].

The significance of this result lies in the role of context selection. The language model's task remains inconsistency detection, but the information presented for each analysis is modified by an earlier processing stage. Consequently, changes in detection performance cannot be attributed solely to the language model itself; the organization of its input also contributes to the observed results. However, the proposed clustering strategy operates on requirements documents and does not investigate context selected through verified cross-artifact traceability relationships [36].

Other research has focused on combining language models with structured requirements and process representations. In 2025, Li and Zheng [39] proposed an LLM-assisted, test-driven framework that transforms natural-language requirements into structured use-case specifications. The framework incorporates business objects and uses consistency rules to validate relationships between use-case flows, UML activity diagrams, and state-machine models [39].

An important characteristic of this approach is that the LLM is primarily responsible for supporting requirements structuring and refinement, while explicit rules and verification algorithms are used to evaluate logical relationships between the resulting representations. This division of responsibilities helps reduce dependence on unconstrained LLM judgments. At the same time, the study combines

artifact generationhuman refinementconsistency validationand test-case generation within one framework [39]. Consequently, its results do not directly isolate the effect of contextual information on checking the consistency of independently created artifacts.

The representation of business process models has also been investigated in LLM-assisted coherence analysis. Schulte et al. [6] introduced aProCheCk in 2025 to support coherence checking across multi-level business process documentation. The method includes preprocessing, content comparison, and coherence-checking stages. During preprocessingnon-essential visual information is removed from BPMN XML representations before the relevant content is analyzed [6].

The authors observed that reducing irrelevant information in BPMN representations improved output quality and resource efficiency [6]. This finding is particularly relevant to the use of LLMs with structured models: a model file can contain information that is necessary for visual presentation but not for evaluating the underlying process description. The approach therefore illustrates how artifact preprocessing can influence the effectiveness of LLM-based analysis. Nevertheless, aProCheCk focuses on coherence within multi-level business process documentation rather than controlled pairwise comparisons between functional requirements, use casesand BPMN models.

A different form of heterogeneous artifact analysis was explored by Dietrich et al. in 2025 [7]. The authors proposed an LLM-based consistency verification approach for programming exercises consisting of problem statements, code templates, reference solutions, and test suites. The framework uses an artifact ontology to organize relationships and distinguish categories of inconsistencies. The implemented verification pipeline examines structural and semantic issues across these representations [7].

The study also evaluates consistency checking on deliberately modified artifact variants containing annotated inconsistencies [7]. This experimental design allows detection results to be compared against known issues. However, the artifacts represent programming exercise specifications and implementationsrather than business-analysis and requirements engineering models. The work therefore provides evidence for ontology-supported multi-artifact verification, while leaving open its applicability to the behavioral relationships of functional requirements, use cases, and BPMN.

Research published in 2026 further developed hybrid approaches combining LLMs with explicit model relationships. Sultan et al. [40] investigated consistency between state machines, use cases, and block diagrams. Their framework combines LLM-based inconsistency detection with dependency graphs used to check selected logical relationships between model elements. The approach also supports correction using language models constrained by formal rules [40].

This study is particularly close to the problem of heterogeneous artifact consistency, as it involves multiple modeling representations and uses their relationships during analysis. However, the authors report that the LLM-based component still produces false-positive and false-negative results [40]. The use of dependency graphs and formal constraints therefore does not remove all uncertainty associated with LLM judgments. Moreover, the investigated artifact combination differs from the functional requirements, use cases, and BPMN models considered in the present study.

Another hybrid approach was proposed by Liu et al. in 2026 [41]. Their method introduces observables as a common abstraction for properties constrained by requirements and models. LLM-based extraction supported by retrieval-augmented generation is used to identify relevant domain information. An ontol-

ogy then supports the harmonization of extracted observables, while the resulting constraints are translated into formulas for solver-based consistency checking [41].

The distinction between semantic interpretation and formal verification is central to this method. The LLM helps transform heterogeneous information into a shared representationbut the final consistency assessment is performed using formal constraints. This architecture provides a different mechanism for handling semantic discrepancies than approaches that ask an LLM to classify inconsistencies directly. However, the method's effectiveness depends on the extraction and formalization of suitable observables, and it does not evaluate the comparative effect of local and traceability-guided prompt context [41].

The importance of domain-specific context has also been investigated in requirements-related LLM analysis. Poulsen et al. [37] compared domain-agnostic and context-aware configurations for requirements ambiguity detection. Their findings indicated that incorporating domain knowledge helped the model distinguish genuinely ambiguous requirements from statements whose interpretation depended on project-specific information [37]. Although ambiguity detection differs from consistency checking, the study provides additional evidence that the information available to an LLM can affect its interpretation of requirements.

Taken together, the reviewed studies illustrate several distinct mechanisms for LLM-assisted consistency analysis. Direct LLM classification offers flexibility when processing natural-language statements [38]. Logical procedures can structure contradiction detection [5], while clustering and preprocessing can modify the information considered during analysis [6], [36]. Structured use cases, ontologies, dependency graphs, and formal verification provide additional mechanisms for representing and checking relationships between heterogeneous artifacts [7],

[39], [40], [41].

These approaches demonstrate that LLM-based consistency checking is not a single method, but a collection of techniques that distribute semantic interpretation, context selection, and verification across different components. Nevertheless, the reviewed literature does not establish whether an LLM's ability to detect behavioral inconsistencies between functional requirements, use cases, and BPMN models improves when its input is enriched through local structural context or validated traceability relationships. This question remains distinct from both automatic link recovery and formal verificationand motivates a controlled comparison of context configurations.

2.5 Research Synthesis and Gap

The reviewed literature reveals two closely related but distinct research directions: the recovery of traceability relationships and the verification of consistency between related software artifacts. Research on automated traceability has progressed from information retrieval techniques toward methods incorporating structural information, learned semantic representations, and retrieval-augmented language models [20], [21], [24], [25], [27]. However, the primary objective of these methods remains the identification of valid correspondences between artifact elements. Establishing such a correspondence does not determine whether the connected elements express mutually compatible behavior.

Existing research has demonstrated that artifact correspondences can support inconsistency detection. Van der Aa et al. [16] identified missing activities and conflicting execution orders between textual process descriptions and process models. Sànchez-Ferreres et al. [32] subsequently investigated more comprehen-

sive alignments involving activities, events, and gateways. In a different artifact setting, Keim et al. [4] used recovered traceability links to detect missing or undocumented elements between software architecture documentation and models. These studies demonstrate that consistency analysis can build upon established relationships. Nevertheless, their methods focus on particular artifact combinations and discrepancy types, and do not directly establish how LLM-based behavioral consistency checking benefits from different sources of contextual information.

The development of LLM-based requirements analysis has expanded the range of techniques available for semantic consistency checking. Early evaluations investigated the direct detection of conflicting requirements using language models [38]. Subsequent approaches introduced formal logic, structured requirements, dependency graphs, ontologies, and external verification mechanisms [5], [39], [40], [41]. Research has also addressed coherence across multi-level business process documentation [6] and consistency between heterogeneous programming exercise artifacts [7].

These contributions demonstrate that LLM-assisted cross-artifact consistency checking is an established research direction. However, the approaches differ in the artifacts being compared, the information available to the model, and the mechanisms used to determine consistency. In some methods, the LLM directly evaluates potential contradictions, whereas in others it extracts or structures information that is subsequently checked by formal rules or solvers [5], [39], [40], [41]. Consequently, their results cannot be interpreted as evidence that a particular configuration of LLM input context will be effective for every combination of artifact types.

The selection and organization of contextual information represent a further unresolved concern. Earlier traceability research demonstrated that structural

information could improve link recovery when its use was guided by validated relationships [24]. Research involving BPMN models similarly showed that selecting relevant execution information was preferable to indiscriminately expanding model fragments [29]. More recently, Fantechi et al. [36] investigated the grouping of requirements before LLM-based inconsistency detection, while Poulsen et al. [37] studied the effects of domain context on requirements ambiguity detection.

Taken together, these findings suggest that the usefulness of additional information depends on its relevance to the analysis being performed. However, they concern different tasks, including traceability recovery, requirements inconsistency detection, and ambiguity analysis. They therefore do not directly establish whether the same contextual information will improve LLM-based consistency checking between independently authored functional requirements, use cases, and BPMN models.

A distinction can be drawn between local structural context and traceability-guided context. Local structural context supports the interpretation of an artifact element within its own representation, whereas traceability-guided context introduces information from other artifact elements connected through established cross-artifact relationships. Both may provide information necessary for interpreting behavioral obligations, but they differ in the origin and organization of that information. Furthermore, supplementary context may contain relevant constraints or unrelated details, making its effect on consistency judgments an empirical question rather than a guaranteed improvement.

Within the reviewed literature, the comparative effect of these context sources has not been established for LLM-based behavioral consistency checking across functional requirements, use cases, and BPMN models. In particular, the examined studies do not isolate the difference between analyzing a linked artifact pair

alone, supplementing it with local structural context, and providing additional information selected through validated traceability relationships.

The present study addresses this gap by examining traceability as a source of context for semantic consistency analysis rather than as an object of automatic link recovery. Using validated correspondences as inputs makes it possible to investigate contextualization without combining its effects with errors introduced during link prediction. The comparison focuses on behavioral inconsistency detection across different artifact pairs and examines both the quality and stability of LLM judgments.

The expected contribution is therefore not a new traceability recovery algorithm or the first method for cross-artifact consistency checking. Rather, it is an empirical assessment of when and to what extent traceability-guided context supports LLM-based semantic consistency verification across heterogeneous analysis artifacts.

Chapter 3

Methodology

...

Chapter 4

Implementation

...

Chapter 5

Evaluation and Discussion

...

Chapter 6

Conclusion

...

Bibliography cited

[1] ISO/IEC/IEEE, *ISO/IEC/IEEE 29148:2018: Systems and software engineering – Life cycle processes – Requirements engineering*, 2018. [Online]. Available: https://www.iso.org/standard/72089.html

[2] B. Ramesh and M. Jarke, "Toward reference models for requirements traceability," *IEEE Transactions on Software Engineering*, vol. 27, no. 1, pp. 58–93, 2001. DOI: 10.1109/32.895989

[3] W. Torres, M. G. J. van den Brand, and A. Serebrenik, "A systematic literature review of cross-domain model consistency checking by model management tools," *Software and Systems Modeling*, vol. 20, pp. 897–916, 2021. DOI: 10.1007/s10270-020-00834-1

[4] J. Keim, S. Corallo, D. Fuchß, and A. Koziolek, "Detecting inconsistencies in software architecture documentation using traceability link recovery," in *2023 IEEE 20th International Conference on Software Architecture (ICSA)*, 2023, pp. 141–152. DOI: 10.1109/ICSA56044.2023.00021

[5] A. E. Gärtner and D. Göhlich, "Automated requirement contradiction detection through formal logic and LLMs," *Automated Software Engineering*, vol. 31, 2024, Art. no. 49. DOI: 10.1007/s10515-024-00452-x

[6] M. Schulte, S. Franzoi, F. Köhne, and J. vom Brocke, "Toward LLM-enabled business process coherence checking based on multi-level process documentation," *Process Science*, vol. 2, 2025, Art. no. 22. DOI: 10.1007/s44311-025-00024-6

[7] F. T. J. Dietrich, Y. Zhou, T. Wasner, S. Krusche, and M. Acosta, "LLM-based multi-artifact consistency verification for programming exercise quality assurance," in *Proceedings of the 25th International Conference on Computing Education Research (Koli Calling 2025)*, 2025. DOI: 10.1145/3769994.3770042

[8] O. C. Z. Gotel and A. Finkelstein, "An analysis of the requirements traceability problem," in *Proceedings of the First IEEE International Conference on Requirements Engineering*, IEEE, 1994, pp. 94–101. DOI: 10.1109/ICRE.1994.292398 [Online]. Available: https://doi.org/10.1109/ICRE.1994.292398

[9] J. Cleland-Huang, O. C. Z. Gotel, J. H. Hayes, P. Mäder, and A. Zisman, "Software traceability: Trends and future directions," in *Proceedings of the on Future of Software Engineering*, ACM, 2014, pp. 55–69. DOI: 10.1145/2593882.2593891 [Online]. Available: https://doi.org/10.1145/2593882.2593891

[10] S. Winkler and J. von Pilgrim, "A survey of traceability in requirements engineering and model-driven development," *Software and Systems Modeling*, vol. 9, pp. 529–565, 2010. DOI: 10.1007/s10270-009-0145-0 [Online]. Available: https://doi.org/10.1007/s10270-009-0145-0

[11] J. Cleland-Huang, O. Gotel, and A. Zisman, Eds., *Software and Systems Traceability*. London: Springer, 2012, ISBN: 978-1-4471-2238-8. DOI:

10.1007/978-1-4471-2239-5 [Online]. Available: https://doi.org/10.1007/978-1-4471-2239-5

[12] P. Mäder and O. Gotel, "Towards automated traceability maintenance," *Journal of Systems and Software*, vol. 85, no. 10, pp. 2205–2227, 2012. DOI: 10.1016/j.jss.2011.10.023 [Online]. Available: https://doi.org/10.1016/j.jss.2011.10.023

[13] M. Ruiz, J. Y. Hu, and F. Dalpiaz, "Why don't we trace? a study on the barriers to software traceability in practice," *Requirements Engineering*, vol. 28, pp. 619–637, 2023. DOI: 10.1007/s00766-023-00408-9 [Online]. Available: https://doi.org/10.1007/s00766-023-00408-9

[14] S. Maro, J.-P. Steghöfer, J. H. Hayes, J. Cleland-Huang, and M. Staron, "Vetting automatically generated trace links: What information is useful to human analysts?" In *2018 IEEE 26th International Requirements Engineering Conference*, IEEE, 2018, pp. 52–63. DOI: 10.1109/RE.2018.00-52 [Online]. Available: https://doi.org/10.1109/RE.2018.00-52

[15] D. Mosquera, M. Ruiz, and O. Pastor, "Ontology-based NLP tool for tracing software requirements and conceptual models: An empirical study," *Requirements Engineering*, vol. 30, pp. 341–369, 2025. DOI: 10.1007/s00766-025-00447-4 [Online]. Available: https://doi.org/10.1007/s00766-025-00447-4

[16] H. van der Aa, H. Leopold, and H. A. Reijers, "Comparing textual descriptions to process models – the automatic detection of inconsistencies," *Information Systems*, vol. 64, pp. 447–460, 2017. DOI: 10.1016/j.is.2016.07.010 [Online]. Available: https://doi.org/10.1016/j.is.2016.07.010

[17] M. Borg, P. Runeson, and A. Ardö, "Recovering from a decade: A systematic mapping of information retrieval approaches to software traceability," *Empirical Software Engineering*, vol. 19, no. 6, pp. 1565–1616, 2014. DOI: 10.1007/s10664-013-9255-y [Online]. Available: https://doi.org/10.1007/s10664-013-9255-y

[18] H. Wan, X. He, Y. Deng, and B. Wang, "A systematic mapping study of information retrieval-based requirements traceability methods," *Information Processing & Management*, vol. 62, no. 6, p. 104 287, 2025. DOI: 10.1016/j.ipm.2025.104287

[19] R. Rasiman, F. Dalpiaz, and S. España, "How effective is automated trace link recovery in model-driven development?" In *Requirements Engineering: Foundation for Software Quality*, 2022, pp. 35–51. DOI: 10.1007/978-3-030-98464-9_4

[20] D. Fuchß et al., "LiSSA: Toward generic traceability link recovery through retrieval-augmented generation," in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, 2025, pp. 1396–1408. DOI: 10.1109/ICSE55347.2025.00186

[21] G. Antoniol, G. Canfora, G. Casazza, A. D. Lucia, and E. Merlo, "Recovering traceability links between code and documentation," *IEEE Transactions on Software Engineering*, vol. 28, no. 10, pp. 970–983, 2002. DOI: 10.1109/TSE.2002.1041053 [Online]. Available: https://doi.org/10.1109/TSE.2002.1041053

[22] A. Marcus and J. I. Maletic, "Recovering documentation-to-source-code traceability links using latent semantic indexing," in *Proceedings of the 25th International Conference on Software Engineering*, IEEE, 2003, pp. 125–

137. DOI: 10.1109/ICSE.2003.1201194 [Online]. Available: https://doi.org/10.1109/ICSE.2003.1201194

[23] J. H. Hayes, A. Dekhtyar, and S. K. Sundaram, "Advancing candidate link generation for requirements tracing: The study of methods," *IEEE Transactions on Software Engineering*, vol. 32, no. 1, pp. 4–19, 2006. DOI: 10.1109/TSE.2006.3 [Online]. Available: https://doi.org/10.1109/TSE.2006.3

[24] A. Panichella et al., "When and how using structural information to improve IR-based traceability recovery," in *2013 17th European Conference on Software Maintenance and Reengineering*, IEEE, 2013. DOI: 10.1109/CSMR.2013.29 [Online]. Available: https://doi.org/10.1109/CSMR.2013.29

[25] J. Guo, J. Cheng, and J. Cleland-Huang, "Semantically enhanced software traceability using deep learning techniques," in *Proceedings of the 39th International Conference on Software Engineering*, IEEE, 2017, pp. 3–14. DOI: 10.1109/ICSE.2017.9 [Online]. Available: https://doi.org/10.1109/ICSE.2017.9

[26] M. Rath, J. Rendall, J. L. C. Guo, J. Cleland-Huang, and P. Mäder, "Traceability in the wild: Automatically augmenting incomplete trace links," in *Proceedings of the 40th International Conference on Software Engineering*, 2018, pp. 834–845. DOI: 10.1145/3180155.3180207

[27] J. Lin, Y. Liu, Q. Zeng, M. Jiang, and J. Cleland-Huang, "Traceability transformed: Generating more accurate links with pre-trained BERT models," in *2021 IEEE/ACM 43rd International Conference on Software Engineering*,

IEEE, 2021, pp. 324–335. DOI: 10.1109/ICSE43902.2021.00040 [Online]. Available: https://doi.org/10.1109/ICSE43902.2021.00040

[28] R. Lapeña, F. Pérez, C. Cetina, and Ó. Pastor, "Leveraging bpmn particularities to improve traceability links recovery among requirements and bpmn models," *Requirements Engineering*, 2022. DOI: 10.1007/s00766-021-00365-1

[29] R. Lapeña, F. Pérez, Ó. Pastor, and C. Cetina, "Leveraging execution traces to enhance traceability links recovery in bpmn models," *Information and Software Technology*, vol. 146, p. 106 873, 2022. DOI: 10.1016/j.infsof.2022.106873

[30] M. Bonner, M. Zeller, G. Schulz, and A. Savu, "LLM-based approach to automatically establish traceability between requirements and MBSE," *INCOSE International Symposium*, vol. 34, no. 1, pp. 2542–2560, 2024. DOI: 10.1002/iis2.13285 [Online]. Available: https://doi.org/10.1002/iis2.13285

[31] U. Thayasivam, K. Verma, A. Kass, and R. Vasquez, "Automatically mapping natural language requirements to domain-specific process models," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 25, 2011, pp. 1695–1700. DOI: 10.1609/aaai.v25i2.18863 [Online]. Available: https://doi.org/10.1609/aaai.v25i2.18863

[32] J. Sànchez-Ferreres, H. van der Aa, J. Carmona, and L. Padró, "Aligning textual and model-based process descriptions," *Data & Knowledge Engineering*, vol. 118, pp. 25–40, 2018. DOI: 10.1016/j.datak.2018.09.001 [Online]. Available: https://doi.org/10.1016/j.datak.2018.09.001

[33] H. van der Aa, H. Leopold, and H. A. Reijers, "Checking process compliance against natural language specifications using behavioral spaces," *Information Systems*, vol. 78, 2018. DOI: 10.1016/j.is.2018.01.007 [Online]. Available: https://doi.org/10.1016/j.is.2018.01.007

[34] B. Aysolmaz, H. Leopold, H. A. Reijers, and O. Demirörs, "A semi-automated approach for generating natural language requirements documents based on business process models," *Information and Software Technology*, vol. 93, pp. 14–29, 2018. DOI: 10.1016/j.infsof.2017.08.009 [Online]. Available: https://doi.org/10.1016/j.infsof.2017.08.009

[35] F. U. Muram, H. Tran, and U. Zdun, "Systematic review of software behavioral model consistency checking," *ACM Computing Surveys*, vol. 50, no. 2, 2017. DOI: 10.1145/3037755 [Online]. Available: https://doi.org/10.1145/3037755

[36] A. Fantechi, S. Gnesi, and L. Semini, "Combining established and emerging techniques to detect inconsistencies in requirements," in *2025 IEEE 33rd International Requirements Engineering Conference (RE)*, 2025, pp. 519–526. DOI: 10.1109/RE63999.2025.00062

[37] V. V. Poulsen, M. Guertler, B. Eisenbart, L. Tomidei, and N. Sick, "Context-aware large language models for ambiguity detection in requirements," in *Proceedings of the DESIGN 2026 19th International Design Conference*, 2026, pp. 2493–2502. DOI: 10.1017/pds.2026.10607

[38] A. Fantechi, S. Gnesi, L. Passaro, and L. Semini, "Inconsistency detection in natural language requirements using ChatGPT: A preliminary evaluation," in *2023 IEEE 31st International Requirements Engineering Conference*,

IEEE, 2023. DOI: 10.1109/RE57278.2023.00045 [Online]. Available: https://doi.org/10.1109/RE57278.2023.00045

[39] H. Li and L. Zheng, "Enhancing requirements via structured formalization and process-state consistency validation: An llm-assisted test-driven framework," *IET Software*, vol. 2025, no. 1, p. 6 714 956, 2025. DOI: 10.1049/sfw2/6714956

[40] B. Sultan, L. Apvrille, and S. Coudert, "On the consistency of state machines, use cases and block diagrams using dependency graphs and large language models," *Software and Systems Modeling*, 2026. DOI: 10.1007/s10270-026-01388-4

[41] T. Liu, S. Tyszberowicz, and B. Beckert, "Observable consistency checking across requirements and models," in *Proceedings of the 21st International Conference on Evaluation of Novel Approaches to Software Engineering (ENASE)*, 2026, pp. 50–63. DOI: 10.5220/0014719400004015